Assignment 1 (24b3003)

1.1 State Machines Under Pressure

(a)

When the timer interrupt occurs:

1. CPU switches to kernel mode.
2. Current process registers and PC are saved in the PCB.
3. Running process changes from Running → Ready.
4. Process is inserted into the ready queue.
5. Scheduler selects another process from the run queue.
6. Context of selected process is restored.
7. Selected process changes Ready → Running.

Data structures used:

- PCB
- Ready queue / run queue
- Saved context / trap frame

(b)(i)

Given sequence:

$$Running \rightarrow Terminated \rightarrow Blocked \rightarrow Ready \rightarrow Running$$

Invalid transitions:

- Running → Terminated
  `read()` on empty socket does not terminate process.
- Terminated → Blocked
  A terminated process cannot become blocked.

(b)(ii)

Correct sequence:

$$Running \rightarrow Blocked \rightarrow Ready \rightarrow Running$$

- Running → Blocked:
  Process calls `read()`.
- Blocked → Ready:
  Network interrupt signals data arrival.
- Ready → Running:
  Scheduler dispatches process.

(c)

A zombie keeps a process table entry so the parent can read its exit status.
Removed using:

`wait()`

If parent exits first:

- zombie is adopted by `init/systemd`
- later cleaned automatically

1.2 Deep fork() Analysis

(a)

For:

```
for(i=0;i<n;i++)
    fork();
```

Total processes:

$$2^n$$

Reason: Each fork doubles the number of processes.

In this program:

- Fork B happens only in child of Fork A
- Parent does not execute Fork B

Total processes:

$$3$$

(b)

Output:

```
C: g=110 x=105
B: g=10 x=5
A: g=10 x=5
```

Explanation:

- Process C modifies its own copy.
- Parent and child have separate memory after `fork()`.

(c)

Ordering:

$$C \to B \to A$$

Impossible:

- A before B
- A before C
- B before C

because both parents call `wait()`.

(d)

(i)

No. Heap memory is copied using copy-on-write after `fork()`.

(ii)

Possible bug:

- use-after-free

Detection:

- Valgrind
- AddressSanitizer

(iii)

If B exits before waiting for C:

- C becomes orphan
- adopted by `init/systemd`

Repeated cases may create zombie accumulation.

2.1 Multi-Process Workload Analysis

Process	Arrival	Burst	Priority
P1	0	10	3
P2	2	4	1
P3	4	6	2
P4	6	2	4
P5	8	8	2

(a)(i)

Preemptions in SRTF:

- At $t = 2$: P1 preempted by P2
- At $t = 6$: P3 preempted by P4

(a)(ii)

Yes. P1 can finish last because shorter jobs keep arriving and preempting it.

(b)

SJF gives lower average wait than FCFS because short jobs do not wait behind long P1.

(c) Round Robin ($q = 3$)

Execution:

```
0-3  P1
3-6  P2
6-9  P1
9-12 P3
12-14 P2
14-16 P4
16-19 P5
19-22 P1
22-25 P3
25-28 P5
```

```
28-29 P1
29-31 P5
```

Turnaround Time:

$$\frac{29 + 12 + 21 + 10 + 23}{5} = 19$$

Wait Time:

$$\frac{19 + 8 + 15 + 8 + 15}{5} = 13$$

RR has larger turnaround than SRTF because CPU time is shared fairly instead of finishing shortest jobs first.

(d) Non-preemptive SJF

Execution:

```
P1: 0-10
P4: 10-12
P2: 12-16
P3: 16-22
P5: 22-30
```

Average wait time:

$$\frac{0 + 4 + 10 + 12 + 14}{5} = 8$$

Average turnaround time:

$$\frac{10 + 6 + 14 + 18 + 22}{5} = 14$$

2.2 Theoretical Bounds

(a)

If:

$$b_1 \leq b_2 \leq b_3$$

SJF order is:

$$b_1, b_2, b_3$$

Average wait time:

$$\frac{0 + b_1 + (b_1 + b_2)}{3} = \frac{2b_1 + b_2}{3}$$

Any other order increases waiting time.

(b)

Under RR:

$$\text{Average response} = \frac{q(k-1)}{2}$$

where:

- q = quantum
- k = number of short jobs

RR improves fairness while SRTF minimizes turnaround.

3.1 Metric Derivation

Process	Start	Finish
P1	0	12
P2	12	15
P3	15	22
P4	22	24
P5	24	30

(a)

Process	WT	TAT
P1	0	12
P2	12	15
P3	15	22
P4	22	24
P5	24	30

(b)

Makespan:

$$30 - 0 = 30$$

(c)

$$J = \frac{73^2}{5(1429)} \approx 0.746$$

(d)

$$J_{SJF} < J_{FCFS}$$

because SJF increases variation in waiting times.

3.2 Real-World Measurements

(a)

Most likely scheduler:

- SJF/SRTF-like scheduler

because it has:

- low mean wait
- very high tail latency

(b)

A scheduler can minimize makespan while increasing average wait by maximizing throughput instead of responsiveness.
Makespan is not ideal for interactive workloads.

(c)

Batch Genomics Pipeline

Choose:

Gamma

because it has lowest makespan.

Real-Time Chat Application

Choose:

Delta

because it has lowest P90 and P99 latency.